Repetition Structures

1. Introduction

A program frequently has to perform the same task over and over again. One way to do this would be simply duplicate the section of code that is repeated as many times as it will be needed. That could make for a very long code, and it also means that if a part of the duplicated code needs to be debugged or amended, the change has to be made multiple times.

A simple way to avoid these problems is to use a **repetition structure** (also called a **loop).**

We will look at two types of loop; **condition controlled loops** which continue to execute while a particular condition is `True` and **count controlled loops** which loop for a set number of times.

2. The `while` loop: A Condition-Controlled Loop

The `while` loop operates a set of statements while a specified condition is `True.`

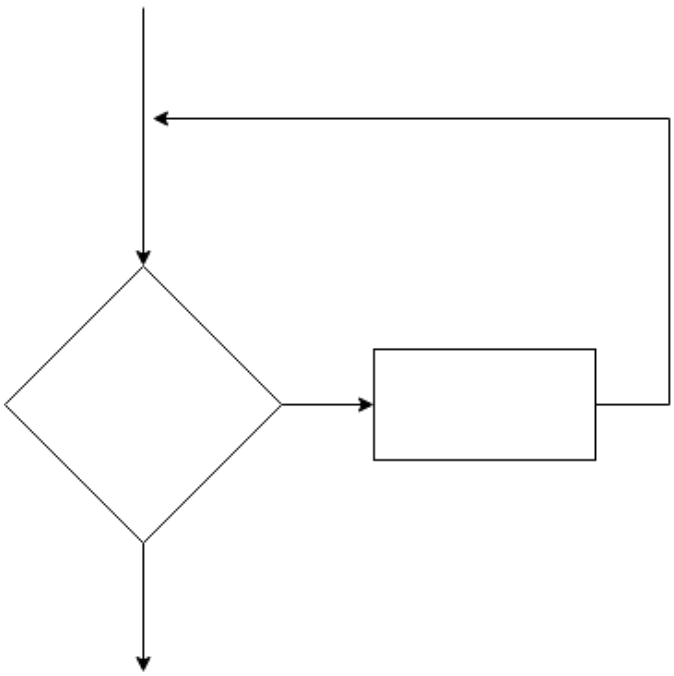

Figure 1 The `while` structure

In Python syntax a `while` loop is written as

```
while condition:
    statement
    statement
    …
Statement
```

Example

commission.py (An edited version of Program 4-1 from Gaddis Starting out with Python)

```
# This program calculates sales commissions.

# Create a variable to control the loop.
keep_going = 'y'

# Calculate a series of commissions.
while keep_going == 'y':
    # Get a salesperson's sales and commission rate.
    sales = float(input('Enter the amount of sales: '))
    comm_rate = float(input('Enter the % commission rate: '))

    # Calculate the commission.
    commission = sales * comm_rate/100

    # Display the commission.
    print('The commission is £', \
          format(commission, '.2f'),sep='')

    # See if the user wants to do another one.
    keep_going = input('Do you want to calculate another ' + \
                       'commission (Enter y for yes): ')
# end of while loop

print('OK - goodbye.')
```

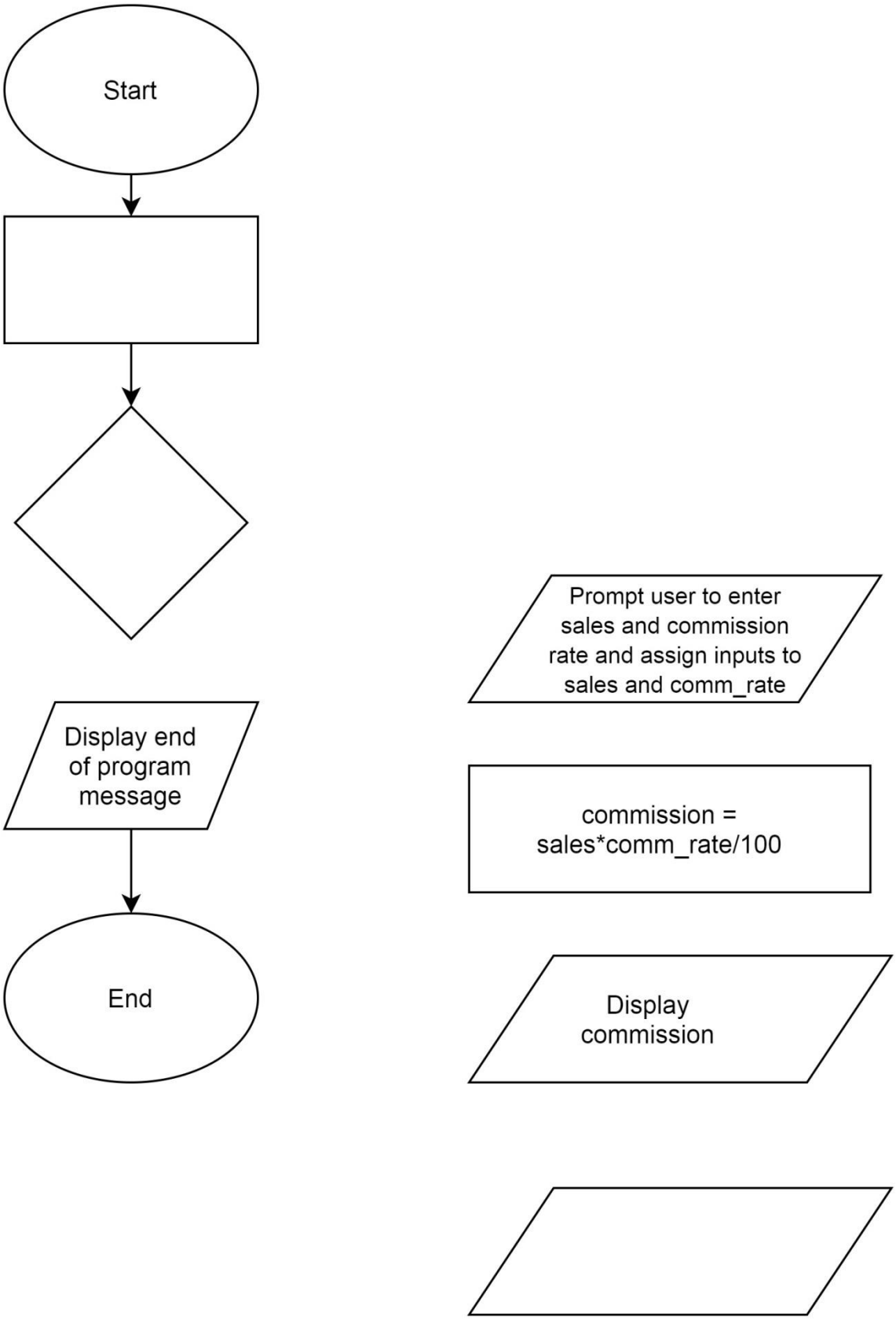

Figure 2 Flow chart for commission.py

3. The `for` Loop: A Count-Controlled Loop

A count-controlled loop iterates a specific number of times.

There are two broad formats in Python for a for loop: the list format and the `range` format.

3.1 The `for loop` using lists

```
for variable in [value1, value2,…]:
    statement
    statement
    …
statement
```

The `for` loop executes as follows. The `variable` is assigned the first value in the list, and the statements in the indented block are executed. The `variable` is then assigned the next value in the list and the indented statements are executed again. This continues until `variable` has been assigned the last value on the list.

Examples

simple_loop2.py

```
# This program  demonstrates a simple for
# loop that uses a list of numbers.

print('Here are the odd numbers from 1 to 11.')
for num in [1, 3, 5, 7, 9, 11]:
    print(num)

# end of for loop
```

simple_loop3.py

```
# This program also demonstrates a simple for
# loop that uses a list of strings.

for name in ['Hippity', 'Flippity', 'Fluppity', 'Flop']:
    print(name)

#end of for loop
```

3.2 The `for` loop using the `range` function

The range function is a very powerful way to write a count controlled loop. It can be used with either one, two or three arguments

The python syntax is

```
for num in range(…):
    statement
    statement
    …
statement
```

Examples

Simple_loop4.py

```
# This program demonstrates how the range
# function can be used with a for loop.
# It simulates parental experience on a
# car journey to the beach.

# Print a message twenty five times.
for x in range(25):
    print('Are we there yet?')

# end of for loop
```

Squares.py

```
# This program uses a loop to display a
# table showing the numbers 1 through 20
# and their squares.

# Print the table headings.
print('Number\tSquare')
print('--------------')

# Print the numbers 1 through 20
# and their squares.
for number in range(1, 21):
    square = number**2
    print(number, '\t', square)

#end of for loop
```

A variant of squares.py

```
# This program uses a loop to display a
# table showing the numbers 20 through to 1
# and their squares.

# Print the table headings.
print('Number\tSquare')
print('---------------')

# Print the numbers 20 through to 1
# and their squares.
for number in range(20, 0, -1):
    square = number**2
    print(number, '\t', square)

#end of for loop
```

4. Augmented Assignment Operators

Quite often in programs we have assignment statements where the variable on the left of the = operator also appears on the right. For example

```
x=x+2
```

What such a statement means is 'take the current value of `x`, add 2 to it, and assign this as the new value of `x'` .

Example

Assuming that **x = 6**, what is the value of x after each of the following statements are performed?

Statement	Value of x after statement
`x=x+4`	
`x=x-5`	
`x=x*5`	
`x=x/3`	
`x=2*x-1`	

These sorts of operations can occur quite often in programming and Python has a set of augmented assignment operators as follows:

Augmented operator	Example	Equivalent to
`+=`	`x += 5`	
`-=`	`y -= 2`	
`*=`	`Z *= 6`	
`/=`	`u /= b`	
`%=`	`c %= 2`	

5. Sentinals

A sentinal is a special value that marks the end of a sequence of items.

When a program reaches a sentinel it knows that the end of a sequence of items has been reached and the loop terminates.

Sentinals should be chosen so as not to be mistaken as part of a regular value in a sequence.

Example

```
# This program displays property taxes.
# It illustrates the use of a sentinel

TAX_FACTOR = 0.075   # Represents the tax factor.

# Get the first lot number.
print('Enter the property lot number')
print('or enter 0 to end.')
lot = int(input('Lot number: '))

# Continue processing as long as the user
# does not enter lot number 0.
while lot != 0:
    # Get the property value.
    value = float(input('Enter the property value in £: '))

    # Calculate the property's tax.
    tax = value * TAX_FACTOR

    # Display the tax.
    print('Property tax: £', format(tax, ',.2f'), sep='')

    # Get the next lot number.
    print('Enter the next lot number or')
    print('enter 0 to end.')
    lot = int(input('Lot number: '))
#end of while loop
```

6. Input Validation Loops

GIGO – Garbage In Garbage Out - is a well-known saying in computer programming. Users will inadvertently find all sorts of ways to enter the ‘wrong sort’ of input. At best this will give Garbage Out. At worst it will crash the program.

Input validation is the process of checking data that has been entered as input to a program to make sure it is valid before it is used in a computation.

This can be done with a loop that iterates for as long as bad data is detected.

Input validation loops are sometimes called **error traps** or **error handlers**.

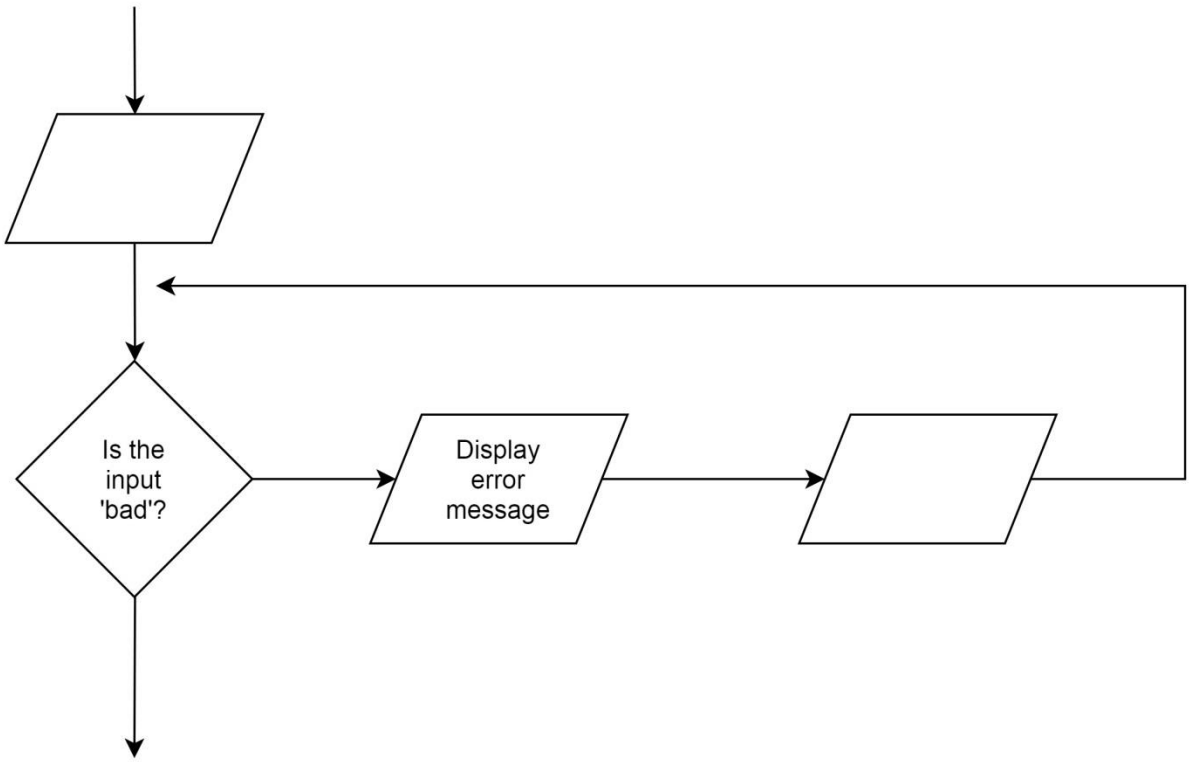

Figure 3 An input validation loop

Example

Retail_with_validation.py

```
# This program calculates retail prices.
# It asks the user for an item's wholesale price
# multiplies this by a set mark_up and displays
# the resulting retail price.
# It then asks the user if s/he would like to enter
# another wholesale price.

# set mark_up value
mark_up = 1.5

# initialise variable to control processing loop.
another = 'y'

# Process one or more items.
while another == 'y' or another == 'Y':
    # Get the item's wholesale cost.
    wholesale = float(input("Enter the item's " + \
                            "wholesale cost: "))

    # Validate the wholesale cost.
    while wholesale < 0:
        print('ERROR: the cost cannot be negative.')
        wholesale = float(input('Enter the correct ' + \
                                'wholesale cost:'))
    # End of validation while loop

    # Calculate the retail price.
    retail = wholesale * mark_up

    # Display the retail price.
    print('Retail price: £', format(retail, ',.2f'))


    # Do this again?
    another = input('Do you have another item? ' + \
                    '(Enter y for yes): ')
# End of while loop for processing items
```

7. Turtle Graphics and Geometric Patterns

A simple way to get turtle to draw a square is via the code,

```
# this program draws a square of side 100

#import turtle
import turtle

# draw a line of length 100
# then turn left thru 90 degrees
# repeat this process 4 times to
# draw the four sides of the square

turtle.forward(100)
turtle.left(90)
turtle.forward(100)
turtle.left(90)
turtle.forward(100)
turtle.left(90)
turtle.forward(100)
turtle.left(90)
```

However, using loops allows us to write the code more efficiently:

```
# this program draws a square of side 100

#import turtle
import turtle

# draw a line of length 100
# then turn left thru 90 degrees
# repeat this process 4 times to
# draw the four sides of the square

for i in range(4):
    turtle.forward(100)
    turtle.left(90)
#end of for loop
```

Supportive Reading

Tony Gaddis, *Starting Out with Python*, 5/e Pearson, 2022, Chapter 4.